

## Domanda 1

Risposta corretta

Punteggio ottenuto 2,00 su 2,00

Yesterday, I met a former student who used to attend the course a dozen years ago and now he is the director of one of the POLIMI departments. He highlighted how things have changed in neural networks since I was teaching "one layer of a (fully connected) feed-forward neural network is sufficient for any task, i.e., regression and classification".

What is true/false about this statement based on your knowledge today?

Note: consider each in the following as independent, but all referring to the quoted sentence above.

| True                             | False                            |                                                                                                                                                                                                                                                 |                                                                                                                                                                                                  |
|----------------------------------|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="radio"/>            | <input checked="" type="radio"/> | Single-layer feed-forward neural networks can solve both regression and classification problems with the desired accuracy. You just need enough S-shaped, i.e., sigmoid or tanh, neurons in that single hidden layer.                           | ✓ The universal approximation theorem states that for classification you need at least two layers                                                                                                |
| <input checked="" type="radio"/> | <input type="radio"/>            | Provided I use enough S-shaped, i.e., sigmoid or tanh, hidden neurons, a single-layer feed-forward neural network can learn any continuous function on a compact set with arbitrary accuracy.                                                   | ✓ True, this is stated in the the universal approximation theorem                                                                                                                                |
| <input type="radio"/>            | <input checked="" type="radio"/> | It has been shown that we can stack as many hidden layers with S-shaped, i.e., sigmoid or tanh, functions as we want and we will always be able to learn the desired approximation for both regression and classification. The more the better! | ✓ If you use S-shaped functions you can incur in the vanishing gradient so you might not learn at all.                                                                                           |
| <input checked="" type="radio"/> | <input type="radio"/>            | We have tasks where the straight application of single-layer neural network to raw data simply does not show any significant robustness or invariance, e.g., image or text classification.                                                      | ✓ This approach has bee proved not to be robust, we need to introduce a feature extractor which is as invariant as possible, such as with CNN where concolutional layers have this specific aim. |

Single-layer feed-forward neural networks can solve both regression and classification problems with the desired accuracy. You just need enough S-shaped, i.e., sigmoid or tanh, neurons in that single hidden layer. : False

Provided I use enough S-shaped, i.e., sigmoid or tanh, hidden neurons, a single-layer feed-forward neural network can learn any continuous function on a compact set with arbitrary accuracy.: True

It has been shown that we can stack as many hidden layers with S-shaped, i.e., sigmoid or tanh, functions as we want and we will always be able to learn the desired approximation for both regression and classification. The more the better!: False

We have tasks where the straight application of single-layer neural network to raw data simply does not show any significant robustness or invariance, e.g., image or text classification.: True

## Domanda 2

Parzialmente corretta

Punteggio ottenuto 0,70 su 2,00

Nowadays feed-forward neural networks are still trained, more or less, as it was done 30 years ago, i.e., via gradient descent methods. Nevertheless, a few significant advancements have been done which have made neural network training more effective.

Select, out of the following, the improvements that have been introduced and have shown to improve the effectiveness of neural network training procedures based on the gradient descent algorithm.

Scegli una o più alternative:

- ☐ a. Attention has been added to seq2seq models in sequence learning
- ☒ b. Rectified linear units and their evolutions, e.g., leaky ReLUs, have replaced sigmoids and tanhs in hidden layers ✔ Yes, this reduces the vanishing gradient issue and speeds up training
- ☒ c. Batch normalization is used to apply normalization at layer level ✔ Yes, this has shown to speed up training because of better conditioning of data
- ☐ d. Skip connections are used whenever it is possible or whenever it makes sense.
- ☒ e. Weight initialization is now based on the number of layer connections such as with Xavier or He initializations. ✔ Yes, this controls the magnitude of the gradient reducing possible issues of exploding gradient
- ☐ f. Long Short-Term Memories have replaced Recurrent Neural Networks in sequential problems
- ☒ g. Convolution and pooling layers are used in image related tasks ✘ No, this is not related to gradient descent, it is related to feature extraction and translation invariance

Your answer is partially correct.

Hai selezionato correttamente 3.

Le risposte corrette sono:

Rectified linear units and their evolutions, e.g., leaky ReLUs, have replaced sigmoids and tanhs in hidden layers ,

Skip connections are used whenever it is possible or whenever it makes sense.,

Weight initialization is now based on the number of layer connections such as with Xavier or He initializations.,

Batch normalization is used to apply normalization at layer level,

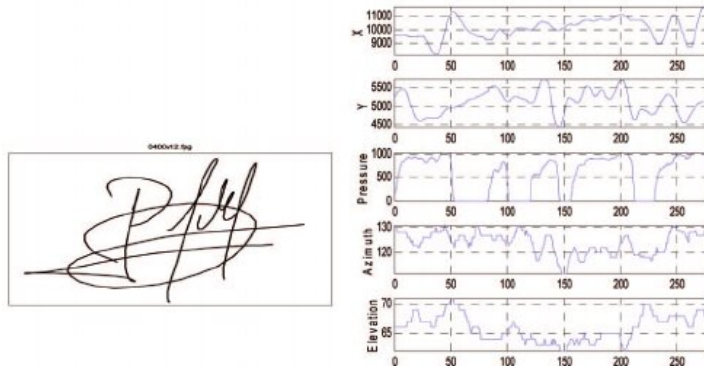
Long Short-Term Memories have replaced Recurrent Neural Networks in sequential problems

## Domanda 3

Completo

Punteggio ottenuto 2,50 su 3,00

Let's consider the task of signature verification from a time series perspective, i.e., let's consider a signature as a multi-variate time sequence. Covariates are: X, Y, Pressure, Azimut, and Elevation.



You work for the New Milan City Bank and you would like to develop a model to tell whether a signature done on a tablet is authentic or not.

- Describe (briefly) 2 different approaches which could be used to recognize if a signature is authentic giving as input a multi-variate time series
- How would you train such models, i.e., which data would you need to train it?

a.1) Possiamo usare le RNN, che basano l'apprendimento su un processo chiamato ricorrenza, in particolare, le RNN sono progettate per processare dati sequenziali e catturare dipendenze all'interno dei dati. Posseggono anche uno stato di memoria interno detto hidden state che genera un output, ad ogni istante viene aggiornato usando l'input attuale e l'output del precedente hidden state. Le RNN possono essere svolte in più istanti, utilizzando un diverso set di pesi e quindi l'hidden state viene aggiornato più volte, catturando dipendenze all'interno dei dati. Tutto ciò permette alle RNN di usare il contesto degli step precedenti e quindi di essere adatte a processare dati sequenziali.

a.2) Possiamo usare le LSTM, che sono progettate per catturare dipendenze a breve e a lungo termine nei dati sequenziali. Questo tipo di NN è composto da:

- Cella di Memoria: dove l'informazione viene immagazzinata.
- Input Gate: controlla quale informazione può scorrere dall'input e arrivare fino alla cella di memoria.
- Forget Gate: controlla quale informazione della cella di memoria deve essere scartata.
- Output Gate: controlla quale informazione della cella di memoria deve scorrere fino ad arrivare all'output.

Osserviamo che tutti i gate sono controllati da un set di pesi e di bias e sono attivati da Sigmoidi, quindi i loro output saranno tra 0 e 1, mentre la cella di memoria è attivata da Tanh, quindi il suo output sarà tra -1 e 1.

b) Potremmo addestrare questi modelli utilizzando le varie covariate fornite, addestrandoli a predire la prossima lettera (o meglio il prossimo "movimento della mano"), in base al contesto degli step precedenti. Ad esempio se la pressione si sta abbassando e tende a 0, è probabile che ci sia una nuova lettera o che la firma sia terminata.

With this problem and input (a 5D time series) approaches which could be used are:

-> Recurrent architectures: RNN, LSTM, BLSTM used for classification ... they are equivalently good and the most obvious to use in this case. I have somehow penalized solutions which have proposed RNN and LSTM (or any other Recurrent networks) as two possible approaches as they are basically the same

-> (1D) Convolutional architectures: Convolutions can be used also with series and they turn out to be quite effective in time series classification. To solve for possibly different length in the sequences we could use a GAP at the end or we could normalize the size of the sequences to be the same by proper interpolation.

These first two approaches are the most natural ones. The training requires positive and negative samples, but these might be difficult to obtain. The most natural way is to use a contrastive loss to build a metric space where signatures are embedded and then new samples are compared to old ones by distance to get if they are true or fake.

<https://proceedings.neurips.cc/paper/1993/file/288cc0ff022877bd3df94bc9360b9c5d-Paper.pdf>

Alternatively, learning could be obtained by asking people to fake a signature of another and setting this up as a classification task. However this might require a user-specific model, i.e., several signatures true/fake for each person, in case a single model for true/fake classification does not work ... as I expect.

Other possibly interesting solutions which still might require a user-specific solution are the following, some are better than others tho:

-> Predictors: a solution that might be quite interesting to try is to use a time series predictor of the signature where a piece of signature is given and the model is asked to predict the rest. The difference between the prediction and the signature is a measure on how different is a new signature from the one the model has seen in the training set. Should be done for each user, but it only requires true samples.

-> Autoencoders: another possible solution that requires to possibly train a model for each user is the use of an autoencoder. In this case, you training a signature embedding and you can use the distance between the embeddings of new signatures from the embeddings of the training signatures as a measure of fake/true. Again should be done for each user, but it requires only true samples at least.

-> GAN: finally we could train a signature generator via GAN then throw away the generator and keep the discriminator of fake/true. Interesting, but too many problems over there. Indeed either you do it for each person or it just learns to generate reasonable time series and it might just compose pieces of different signatures without really noticing the difference. Also generating the signature of a different person might be fine for the discriminator if you put all the samples together.

The grade takes into consideration also the degree to which the proposed solutions have been motivated, described and discussed.

Commento:

## Domanda 4

Completo

Punteggio ottenuto 2,75 su 3,00

With reference to the Neural Turing Machines answer the following questions (use the letters to organize your answer)

- a) Describe the Neural Turing Machine idea and how read / write instructions are implemented
- b) Describe 2 examples of tasks that could be suited for a Neural Turing Machine better than a seq2seq model and motivate your answers

a) Le NTM sono un tipo di NN che emula il comportamento di un TM, in cui abbiamo una stringa di simboli su alla quale possiamo eseguire qualsiasi operazione, andando a leggere o scrivere su di essa. Le NTM sono composte da una CNN e da un banco di memoria esterna, ovvero hanno un controller NN e una matrice di memoria esterna.

Il controller usa il proprio stato di memoria interna per andare a leggere o scrivere sulla memoria esterna della rete, permettendoci di eseguire task che richiedono l'utilizzo di memoria a lungo termine.

Il meccanismo di write delle NTM permette di scrivere sulla memoria esterna della rete, andando a prendere un indirizzo e cambiandone il contenuto. Prima calcola un set di pesi memory-reading, appresi durante il training, che vengono usati per interpolare tra le locazioni di memoria. Poi la rete andrà a calcolare un set di pesi write per poter scrivere sulla memoria esterna delle rete, interpolando tra i pesi di lettura e scrittura con il contenuto attuale e il vecchio contenuto.

Il meccanismo di read, permette di andare a leggere il contenuto della memoria esterna della rete, vengono calcolati un set di pesi read per poter pesare le locazioni di memoria. L'output sarà un vettore che contiene il contenuto della memoria letta e potrà essere usato per fare predizioni o calcoli.

b.1) Le NTM sono più adatte quando è richiesta della memoria a lungo termine, quindi sono più adatte al modello seq2seq se vogliamo eseguire task di traduzione.

b.2) Le NTM sono più adatte quando è richiesta della memoria a lungo termine, quindi sono più adatte al modello seq2seq se vogliamo eseguire task di voice-to-text.

With reference to [https://chrome.deib.polimi.it/images/d/d7/AN2DL\\_06\\_2223\\_BeyondSeq2Seq.pdf](https://chrome.deib.polimi.it/images/d/d7/AN2DL_06_2223_BeyondSeq2Seq.pdf)

a) slides 4 and 5

b) up to you but can take inspiration from slide 7, most common mistakes: non providing real examples, not providing 2 examples, providing two examples which are basically one the epsilon variation of the other

Commento:

## Domanda 5

Risposta errata

Punteggio ottenuto 0,00 su 2,00

The following questions concern Class Activation Mapping, CAM.

Mark all the sentences that are correct.

Please, refrain from marking answers that are "technically possible in Python", and stick to those answers that a Neural Network expert (as you are expected to be) would give and the solution he would adopt.

Scegli una o più alternative:

- ☒ a. CAM are difficult to implement in ResNet-like architectures, given the blocks with skip connections at the the first layers... ✗ FALSE: it is enough to have an output layer with a GAP, no matter what's before that.
- ☐ b. Class Activation Mapping are explainability tools for convolutional neural network.
- ☐ c. CAM suggests that the last layers of a CNN preserve localization properties
- ☒ d. A positive aspect of CAMs is that you do not need to store any network activations, just the GAP outputs ✗ FALSE: you need also GAP inputs to compute the maps. GAP's outputs are just numbers.
- ☒ e. In order to compute CAM, the network needs to have a Global Averaging Pooling Layer before a single dense layer for output. In particular, the number of channels entering the GAP should be the same as the number of classes. ✗ FALSE: There is no need to have the same number of channels as the output neurons. This is only true when classification is performed by softmax after GAP, but CAM uses a dense layer in between.
- ☐ f. CAM are very flexible as they can be used in any CNN classifier of any architecture, without need for retraining.
- ☐ g. CAM has a low computational overhead at inference time
- ☒ h. CAM are typically a lower resolution than the input image, some zooming like bilinear upsampling is required ✓
- ☐ i. CAM are used to boost performance of localization networks
- ☒ j. Class Activation Mapping are often used with AutoEncoders to spot anomalous regions in images. ✗ FALSE: CAM needs to be trained for classification, as we have seend during lectures (yuo need to train the weights of the last dense layers)

Your answer is incorrect.

Le risposte corrette sono:

Class Activation Mapping are explainability tools for convolutional neural network.,

CAM suggests that the last layers of a CNN preserve localization properties,

CAM has a low computational overhead at inference time,

CAM are typically a lower resolution than the input image, some zooming like bilinear upsampling is required

## Domanda 6

Risposta corretta

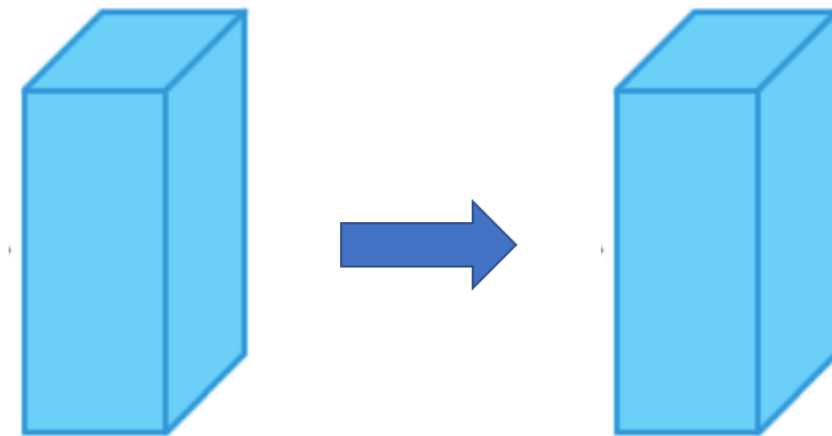
Punteggio ottenuto 1,00 su 1,00

Which of the following layers can be technically plugged in between the activations maps in the following scheme

Lines of code refer to the following import.

```
import tensorflow as tf
tfkl = tf.keras.layers
```

H = 128, W = 128, C = 36



H = 128, W = 128, C = 36

Scegli una o più alternative:

- ☒ a. `tfkl.Conv2D(36, kernel_size=3, padding='same', activation='relu')` ✓ YES!
- ☐ b. `tfkl.UpSampling2D()`
- ☒ c. `layers.Activation(activations.relu)` ✓ YES: activations do not change sizes
- ☐ d. it is not possible to say, since we don't know what will happen in the next layers
- ☐ e. `tfkl.Conv2D(64, kernel_size=1, padding='same', activation='relu')`
- ☐ f. `GlobalAveragingPooling2D()`
- ☒ g. `tfkl.MaxPooling2D(x, pool_size=(3, 3), strides = (1,1), padding = 'same')` ✓ YES: this does not reduce the spatial extent, as the strides is (1,1)

Your answer is correct.

Le risposte corrette sono:

```
tfl.Conv2D(36, kernel_size=3, padding='same', activation='relu')  
,  
  
tfl.MaxPooling2D(x, pool_size=(3, 3), strides = (1,1), padding = 'same')  
,  
  
layers.Activation(activations.relu)
```

**Domanda 7**

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Pooling Layers, please mark only the true answers among the following.

Scegli una o più alternative:

- ☐ a. Pooling layers can be placed everywhere inside a VGG-like CNN
- ☐ b. Pooling layers are the only way to reduce the spatial extent of activation maps
- ☒ c. Pooling layers increases the receptive field ✓
- ☒ d. Pooling layers cannot be used inside a residual block ✓ TRUE: otherwise the spatial dimensions from the two branches do not match
- ☐ e. Pooling layers always need to add a nonlinearity

Your answer is correct.

Le risposte corrette sono:

Pooling layers increases the receptive field,

Pooling layers cannot be used inside a residual block



Domanda **8**

Risposta non data

Non valutata

Consider the following code

```
import tensorflow as tf
tfkl = tf.keras.layers
tfk = tf.keras

def get_model_3(input_shape):

    input_layer = tfkl.Input(shape=input_shape, name='input_layer')

    x = tfkl.Conv2D(64, kernel_size=4, padding='same', strides=(2,2), activation='relu', name='c1')(input_layer)
    x = tfkl.MaxPooling2D(name='mp1')(x)
    x = tfkl.Conv2D(128, kernel_size=4, padding='same', strides=(2,2), activation='relu', name='c2')(x)
    x = tfkl.MaxPooling2D(name='mp2')(x)
    x = tfkl.Conv2D(256, kernel_size=1, padding='same', activation='relu', name='c3')(x)
    x = tfkl.Conv2D(128, kernel_size=1, padding='same', activation='relu', name='c4')(x)
    x = tfkl.Dropout(0.2, name='do1')(x)
    x = tfkl.Conv2D(256, kernel_size=1, padding='same', activation='relu', name='c5')(x)
    x = tfkl.UpSampling2D(name='up2', size=(4,4))(x)
    x = tfkl.Conv2D(128, kernel_size=4, padding='same', activation='relu', name='c6')(x)
    x = tfkl.UpSampling2D(name='up3', size=(4,4))(x)
    x = tfkl.Conv2D(64, kernel_size=4, padding='same', activation='relu', name='c7')(x)

    # Output Layer
    output_layer = tfkl.Conv2D(input_shape[-1], kernel_size=1, padding='same', activation="sigmoid", name='output_layer')(x)

    model = tf.keras.Model(inputs=input_layer, outputs=output_layer)
    model.compile(
        loss = tfk.losses.MeanSquaredError(),
        optimizer = tfk.optimizers.Adam(),
    )
    return model

input_shape = (32,64,3)
model = get_model_3(input_shape=input_shape)
model.summary()
tfk.utils.plot_model(model)
```

The last command returns the model summary. Below you can find a reference for you to work on the model. Please fill in the scheme and upload it. This is a backup option, the following question will be assessed

| Layer (type)             | Output Shape | Param # |
|--------------------------|--------------|---------|
| =====                    |              |         |
| input_layer (InputLayer) |              |         |
| c1 (Conv2D)              |              |         |
| mp1 (MaxPooling2D)       |              |         |
| c2 (Conv2D)              |              |         |
| mp2 (MaxPooling2D)       |              |         |
| c3 (Conv2D)              |              |         |
| c4 (Conv2D)              |              |         |
| do1 (Dropout)            |              |         |
| c5 (Conv2D)              |              |         |

```

up2 (UpSampling2D)

c6 (Conv2D)

up3 (UpSampling2D)

c7 (Conv2D)

output_layer (Conv2D)

```

=====

Total params:

Trainable params:

Non-trainable params:

-----

Model: "model"

| Layer (type)                    | Output Shape        | Param # | Connected to                                         |
|---------------------------------|---------------------|---------|------------------------------------------------------|
| input_layer (InputLayer)        | [(None, 32, 32, 3)] | 0       | []                                                   |
| block_1_conv_1 (Conv2D)         | (None, 32, 32, 64)  | 4864    | ['input_layer[0][0]']                                |
| block_1_mp (MaxPooling2D)       | (None, 16, 16, 64)  | 0       | ['block_1_conv_1[0][0]']                             |
| block_1_dropout (Dropout)       | (None, 16, 16, 64)  | 0       | ['block_1_mp[0][0]']                                 |
| block_1_conv_2 (Conv2D)         | (None, 16, 16, 128) | 204928  | ['block_1_dropout[0][0]']                            |
| block_1_conv_3 (Conv2D)         | (None, 16, 16, 64)  | 204864  | ['block_1_conv_2[0][0]']                             |
| block_1_add (Add)               | (None, 16, 16, 64)  | 0       | ['block_1_dropout[0][0]',<br>'block_1_conv_3[0][0]'] |
| block_2_conv_1 (Conv2D)         | (None, 16, 16, 128) | 204928  | ['block_1_add[0][0]']                                |
| block_2_mp (MaxPooling2D)       | (None, 8, 8, 128)   | 0       | ['block_2_conv_1[0][0]']                             |
| block_2_dropout (Dropout)       | (None, 8, 8, 128)   | 0       | ['block_2_mp[0][0]']                                 |
| block_2_conv_2 (Conv2D)         | (None, 8, 8, 256)   | 819456  | ['block_2_dropout[0][0]']                            |
| block_2_conv_3 (Conv2D)         | (None, 8, 8, 128)   | 819328  | ['block_2_conv_2[0][0]']                             |
| block_2_add (Add)               | (None, 8, 8, 128)   | 0       | ['block_2_dropout[0][0]',<br>'block_2_conv_3[0][0]'] |
| block_2_gap (GlobalAveragePool) | (None, 128)         | 0       | ['block_2_add[0][0]']                                |
| output_layer (Dense)            | (None, 6)           | 774     | ['block_2_gap[0][0]']                                |

=====

Total params: 2,259,142

Trainable params: 2,259,142

Non-trainable params: 0

## Domanda 9

Risposta corretta

Punteggio ottenuto 5,00 su 5,00

Remember that this question will be automatically rated, therefore you have to consider the following details very carefully:

- when the activations become a vector, these are stacked along the third dimension (the channel, C).  
Therefore, if your latent representation becomes a 3 element vector, enter as sizes H:1 W:1 C:3.
- enter digits **only** and do not use "." or "," or "-" or "[]". Say write "1234" and not "1,234" or "1.234". Write "0", not "-" or "[]".
- do not enter formulas, just resulting numbers
- connections, if any have been omitted in this summary

All the numbers have to be entered here, the text in the previous question is just meant for you to design the solution.

| Layer (type)             | Output Shape | Param # |
|--------------------------|--------------|---------|
| =====                    |              |         |
| input_layer (InputLayer) | H:           |         |
| <div>32</div>            |              |         |
| ✓ , W:                   |              |         |
| <div>64</div>            |              |         |
| ✓ , C:                   |              |         |
| <div>3</div>             |              |         |
| ✓ Params #:              |              |         |
| <div>0</div>             |              |         |
| ✓                        |              |         |
| c1 (Conv2D)              | H:           |         |
| <div>16</div>            |              |         |
| ✓ , W:                   |              |         |
| <div>32</div>            |              |         |
| ✓ , C:                   |              |         |
| <div>64</div>            |              |         |
| ✓ Params #:              |              |         |
| <div>3136</div>          |              |         |
| ✓                        |              |         |
| mp1 (MaxPooling2D)       | H:           |         |
| <div>8</div>             |              |         |
| ✓ , W:                   |              |         |
| <div>16</div>            |              |         |
| ✓ , C:                   |              |         |
| <div>64</div>            |              |         |
| ✓ Params #:              |              |         |
| <div>0</div>             |              |         |
| ✓                        |              |         |
| c2 (Conv2D)              | H:           |         |
| <div>4</div>             |              |         |
| ✓ , W:                   |              |         |
| <div>8</div>             |              |         |
| ✓ , C:                   |              |         |
| <div>128</div>           |              |         |
| ✓ Params #:              |              |         |
| <div>131200</div>        |              |         |
| ✓                        |              |         |
| mp2 (MaxPooling2D)       | H:           |         |
| <div>2</div>             |              |         |
| ✓ , W:                   |              |         |
| <div>4</div>             |              |         |
| ✓ , C:                   |              |         |
| <div>128</div>           |              |         |
| ✓ Params #:              |              |         |
| <div>0</div>             |              |         |
| ✓                        |              |         |

c3 (Conv2D) H:

2

✓ , W:

4

✓ , C:

256

✓ Params #:

33024

✓

c4 (Conv2D) H:

2

✓ , W:

4

✓ , C:

128

✓ Params #:

32896

✓

do1 (Dropout) H:

2

✓ , W:

4

✓ , C:

128

✓ Params #:

0

✓

c5 (Conv2D) H:

2

✓ , W:

4

✓ , C:

256

✓ Params #:

33024

✓

up2 (UpSampling2D) H:

8

✓ , W:

16

✓ , C:

256

✓ Params #:

0

✓

c6 (Conv2D) H:

8

✓

, W:

16

✓

, C:

128

✓

Params #:

524416

✓

up3 (UpSampling2D)

H:

32

✓

, W:

64

✓

, C:

128

✓

Params #:

0

✓

c7 (Conv2D)

H:

32

✓

, W:

64

✓

, C:

64

✓

Params #:

131136

✓

output\_layer (Conv2D)

H:

32

✓

, W:

64

✓

, C:

3

✓

Params #:

195

✓

=====

Total params:

889027

✓

Trainable params:

889027

✓

Non-trainable params:

0

✓

| Layer (type) | Output Shape | Param # |
|--------------|--------------|---------|
|--------------|--------------|---------|

=====

|                          |                     |        |
|--------------------------|---------------------|--------|
| input_layer (InputLayer) | [(None, 32, 64, 3)] | 0      |
| c1 (Conv2D)              | (None, 16, 32, 64)  | 3136   |
| mp1 (MaxPooling2D)       | (None, 8, 16, 64)   | 0      |
| c2 (Conv2D)              | (None, 4, 8, 128)   | 131200 |
| mp2 (MaxPooling2D)       | (None, 2, 4, 128)   | 0      |
| c3 (Conv2D)              | (None, 2, 4, 256)   | 33024  |
| c4 (Conv2D)              | (None, 2, 4, 128)   | 32896  |
| do1 (Dropout)            | (None, 2, 4, 128)   | 0      |
| c5 (Conv2D)              | (None, 2, 4, 256)   | 33024  |
| up2 (UpSampling2D)       | (None, 8, 16, 256)  | 0      |
| c6 (Conv2D)              | (None, 8, 16, 128)  | 524416 |
| up3 (UpSampling2D)       | (None, 32, 64, 128) | 0      |
| c7 (Conv2D)              | (None, 32, 64, 64)  | 131136 |
| output_layer (Conv2D)    | (None, 32, 64, 3)   | 195    |

=====

Total params: 889,027

Trainable params: 889,027

Non-trainable params: 0



## Domanda 10

Parzialmente corretta

Punteggio ottenuto 0,30 su 1,00

Mark all the tasks that the above network can be trained for.

Please, refrain from marking answers that are "technically and eventually possible in Python", and stick to those tasks that a Neural Network expert (as you are expected to be) would use this network for.

Needless to say, assume you are always provided with the required training data for these tasks, while you cannot change the architecture and training options for the above model.

Scegli una o più alternative:

- ☐ a. Given a human face as input, estimate its age, weight and length of eyebrows
- ☐ b. Given the picture of a person, estimate the coordinates of the tip of the nose and of the center of left and right hand
- ☒ c. Suppress blur in a grayscale ✗ False, this is not compliant with network dimension, the input / output have three channels
- ☐ d. Classify the the digits from 0 to 2 in a subset of MNIST dataset
- ☐ e. Determine which region of an image is covered by pedestrian, cars or background (i.e., anything else)
- ☐ f. Given a map for whether forecast, estimate the probability of rain, the probability of snow and the probability of strong winds in each pixel.
- ☒ g. Suppress noise in an image ✓ TRUE: that's an autoencoder for an RGB image

Your answer is partially correct.

Hai selezionato correttamente 1.

Le risposte corrette sono:

Suppress noise in an image,

Given a map for whether forecast, estimate the probability of rain, the probability of snow and the probability of strong winds in each pixel.